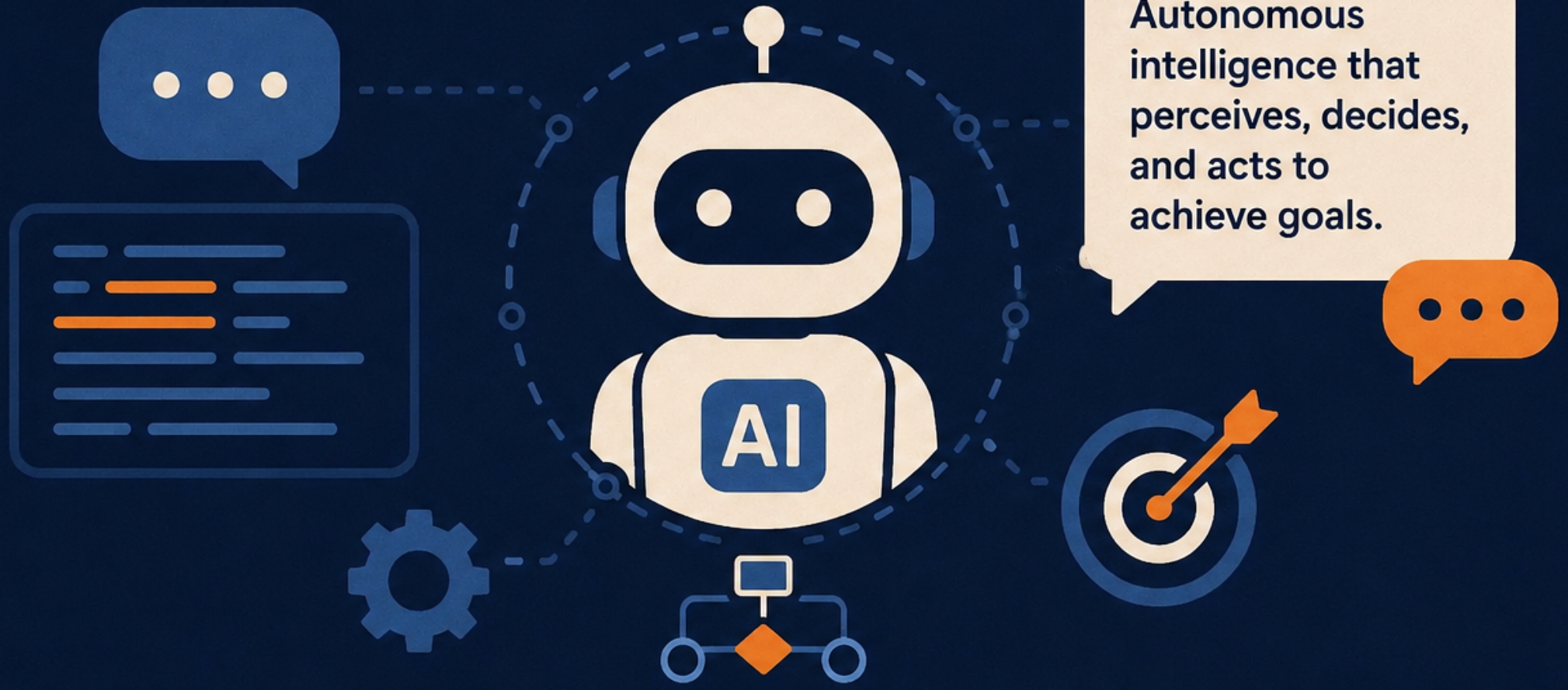
AI AGENTS



PERCEIVE

REASON

DECIDE

ACT

ACHIEVE

Lecture: Building AI agents

#Agenda

1) Recap (Text2Sql)
2) Improving our Text2Sql
3) Agentic Workflow

Components of AI Agent

1) persona → Who they are? For Eg: Expert Math tutorve
2) LLM as the knowledge brain (Planner)
   ↳ Reasoning based
3) prompting strategy
4) Tools

LLM ⟶ Non-deterministic

⇒ "The temperature of Paris is 6°C" free text

↓

structured format

(JSON)

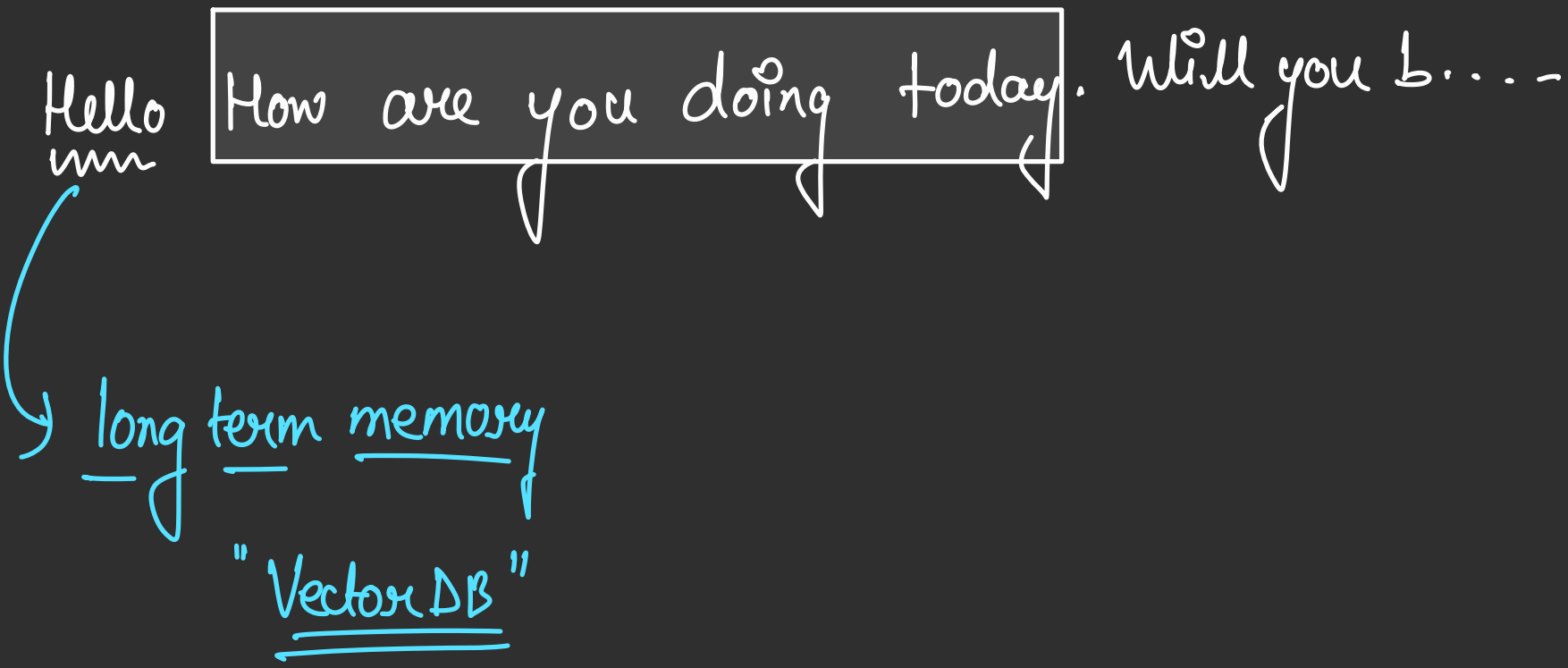
```
{
  place: Paris,
  Temp : 6 ,
  degree : True }
```

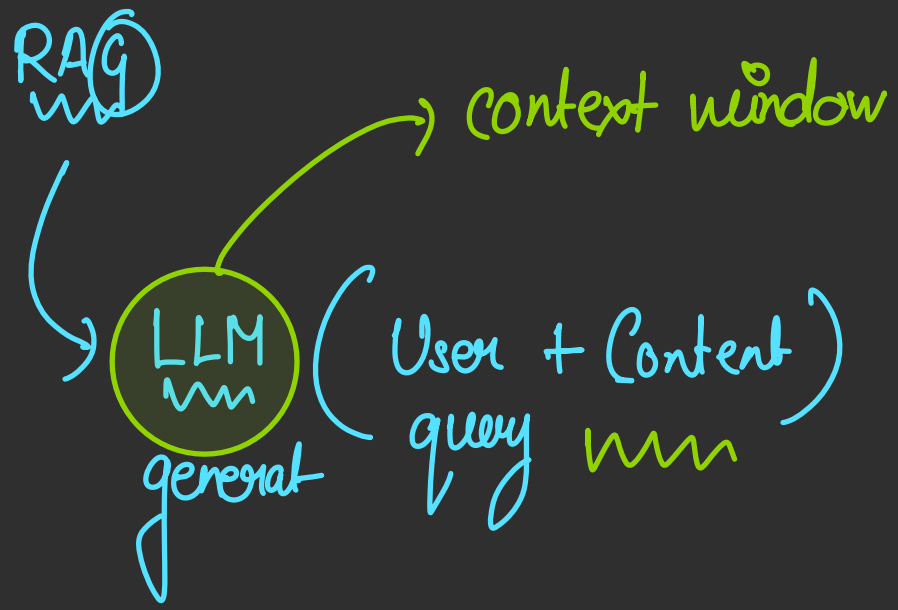
LLM → Context Window (180K)

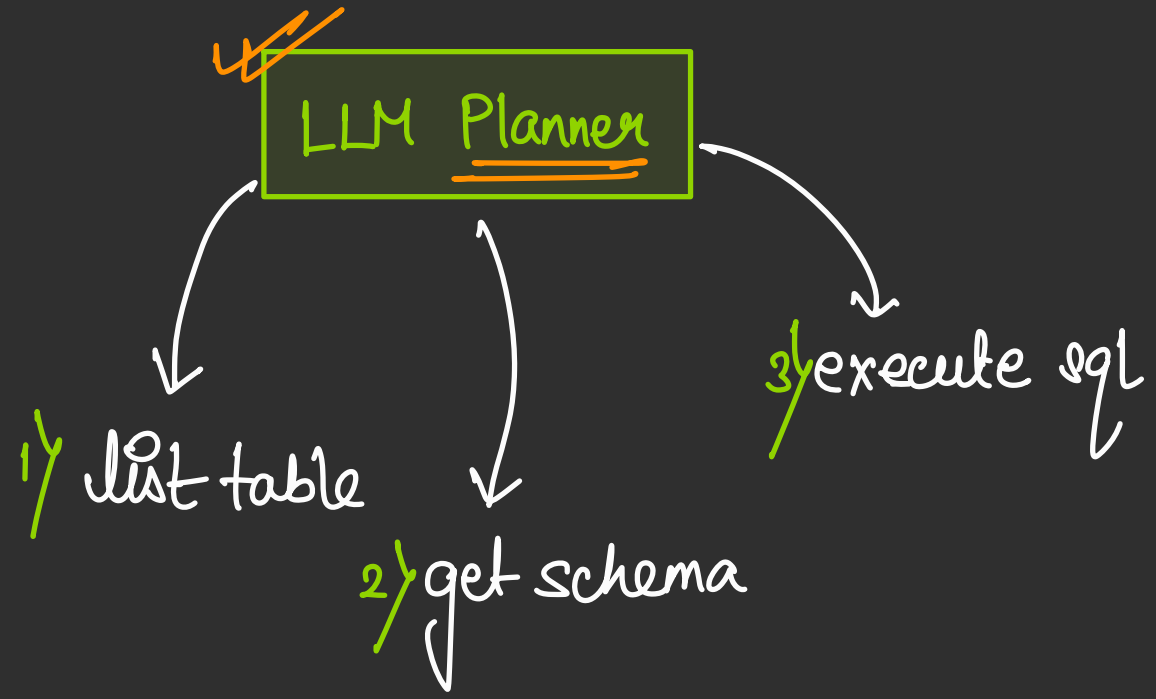
User Query → LLM → response

1) Tool: Tool outputs (RAG) / API / - - - -
2) Conversation history

10 iterat


Hello
How are you doing today. Will you b...
long term memory
"Vector DB"


RAG
context window
LLM
User + Content
query
generat


LLM Planner
1) list table
2) get schema
3) execute sql

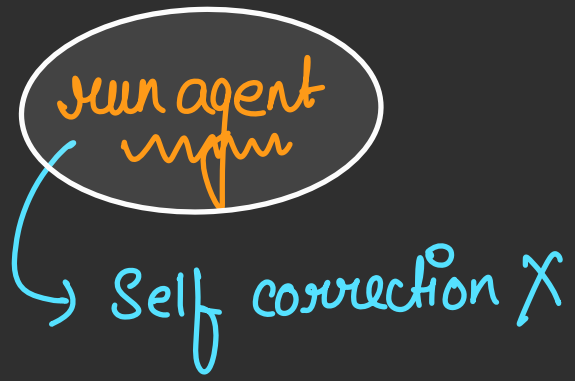
Self-Improvement in Text2SQL Agents: Key Concepts

How agents evaluate, reflect, correct, and refine their own SQL at inference time — without retraining.

COMPONENT	WHAT IT MEANS	HOW IT WORKS	PURPOSE / BENEFITS	EXAMPLE (TEXT2SQL)
1. Self-Reflection (Why it went wrong)	The agent generates a natural-language critique of its own SQL before attempting a fix.	• LLM reads the generated SQL, schema, question, and error. • Explains the likely reason for failure in natural language. • Produces a reflective explanation (semantic gradient).	✓ Provides explicit reasoning. ✓ Turns errors into actionable feedback. ✓ Improves transparency and interpretability.	**Generated SQL:** `SELECT salary FROM employee` **Reflection:** "I used `salary` but the correct column is `annual_salary`."
2. Self-Correction (Targeted fix)	The agent detects a specific defect and applies a precise fix — a surgical repair, not open-ended reasoning.	• Identify the exact error type (syntax, column, table, join, etc.). • Apply minimal changes to repair it. • Re-generate the corrected SQL.	✓ Fast and efficient. ✓ Fixes concrete, local issues. ✓ Reduces unnecessary changes.	**Error:** Column `salary` not found **Correction:** Replace `salary` with `annual_salary`
3. Iterative Refinement (Self-Refine Loop)	The agent repeats the Generate → Feedback → Refine cycle until a stopping condition is met.	• Generate initial SQL. • Get execution / semantic feedback. • Refine SQL using the feedback. • Repeat (typically 2–5 iterations).	✓ Leads to progressive improvement. ✓ Shows 5–40% performance gains (Madaan et al., 2023). ✓ Uses the same LLM (no reward model needed).	**Iteration 1:** Wrong column **Iteration 2:** Correct column, missing JOIN **Iteration 3:** Correct JOIN, valid result
4. Memory-Based Improvement (Learn from past failures)	Past failures and fixes are stored and retrieved to inform future SQL generation.	• Store (failed SQL, error, corrected SQL, reflection, question/schema) tuples. • Retrieve similar past failures. • Inject relevant memories into prompts as few-shot examples.	✓ Converts episodic mistakes into reusable knowledge. ✓ Improves accuracy on similar future queries. ✓ Formalized in Reflexion (2023).	**Memory Entry:** Q: Top paid employees Failed: SELECT `salary` … Error: Column not found Fixed: SELECT `annual_salary` …
5. Feedback-Conditioned Prompting (Inform the next attempt)	The refinement prompt includes original SQL, structured feedback, error messages, and memories.	• Build a rich context: question, schema, SQL, error, reflection, memories. • Ask the LLM to produce an improved SQL. • The feedback conditions a better output.	✓ Each attempt is strictly more informed. ✓ Reduces repeated mistakes. ✓ Improves final SQL quality.	**Prompt Context Includes:** • Original SQL • Error: Column `salary` not found • Reflection: Use `annual_salary` • Similar past fix example → Generate improved SQL

Key Takeaway

Self-improvement in Text2SQL agents is an inference-time loop of reflection, correction, memory, and feedback-conditioned refinement.
It transforms execution feedback into better SQL — without updating model weights or human intervention.



- self correction
- error catching
- memory loging

Introduction to Agentic Workflows

From single-shot prompts to adaptive, goal-driven systems

TOPIC	DESCRIPTION	KEY POINTS	ARCHITECTURAL DISTINCTION	IMPLICATIONS
1. What Are Agentic Workflows?	A system where an LLM actively plans, executes actions, observes results, and adapts behavior to achieve a goal through a structured process with defined stages, control flow, and termination conditions.	**Not just a response generator.** The LLM participates in a process. **Structured, not open-ended.** Follows a workflow with stages and control flow. **Goal-driven and adaptive.** Observes outcomes and adjusts its next steps.	**Workflows vs Agents** **Workflows** Orchestrate LLMs and tools through predefined code paths — the developer decides the sequence. **Agents** The LLM decides which tools to call, in what order, and when to stop. Most production systems are hybrids: deterministic scaffolding + model-driven decisions.	Predictable and controllable execution. Easier to debug, test, and observe. Clear termination conditions. Suitable for production-critical use cases.
2. Spectrum of LLM System Complexity	Agentic workflows sit on a spectrum of increasing capability and complexity — from simple single calls to adaptive, self-improving loops.	**Increasing capability** From single response → structured chain → adaptive loop. **Increasing complexity** More control flow, feedback, and tool use. **Increasing reliability** More chances to get it right through iteration.	**The Spectrum** Single-shot Prompting → Static Chains (Prompt Chains) → Agentic Workflows Send question → get answer. No iteration, no tools, no verification. One chance to be right. Fixed sequence of LLM calls (e.g., outline → draft → edit). No revisiting steps if something fails. Works for predictable tasks. Generate → evaluate → refine / reroute → repeat. The LLM operates in a persistent control loop across steps.	Choose the right level of complexity for the task. Static chains break on unpredictable problems. Workflows handle uncertainty and change.
3. Practical Trade-Off	Agentic systems trade latency and compute for reliability. More iterations = higher cost, but significantly better accuracy and robustness.	**Accuracy improves** with feedback + retries. More LLM calls increase latency and cost. **Engineer the right balance** for the use case.	**Example: SQL Query Generation** **Single LLM Call** ~70% accurate 1x cost / latency → **Agentic Workflow** (Generate → Critique → Retry) ~95% accurate 2–4x cost / latency **Decision Question** Is the reliability gain worth the extra cost for this use case?	More iterations = higher reliability. Cost increases 2–4x typically. Use agentic workflows where correctness matters most.

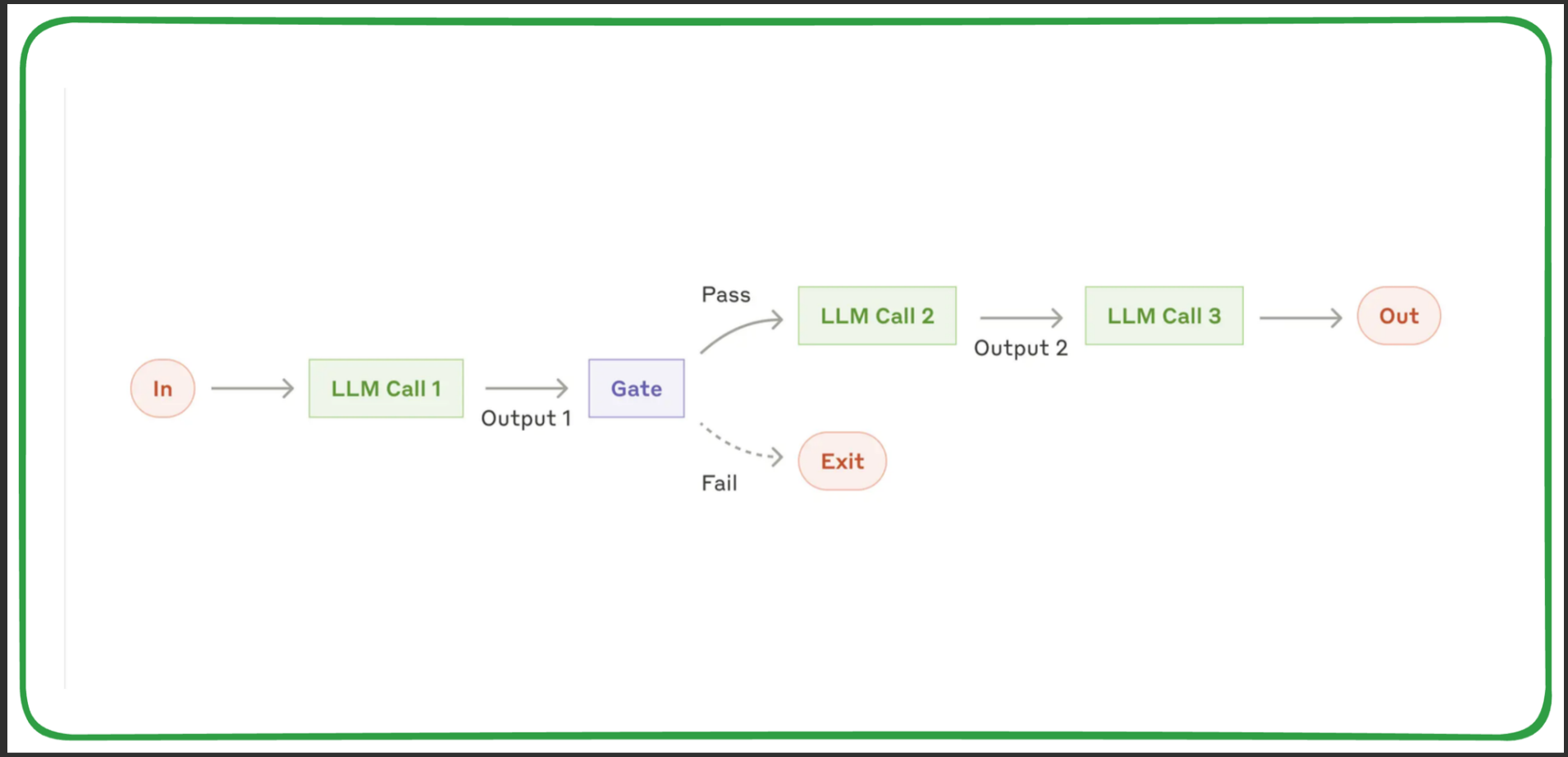
Key Takeaway

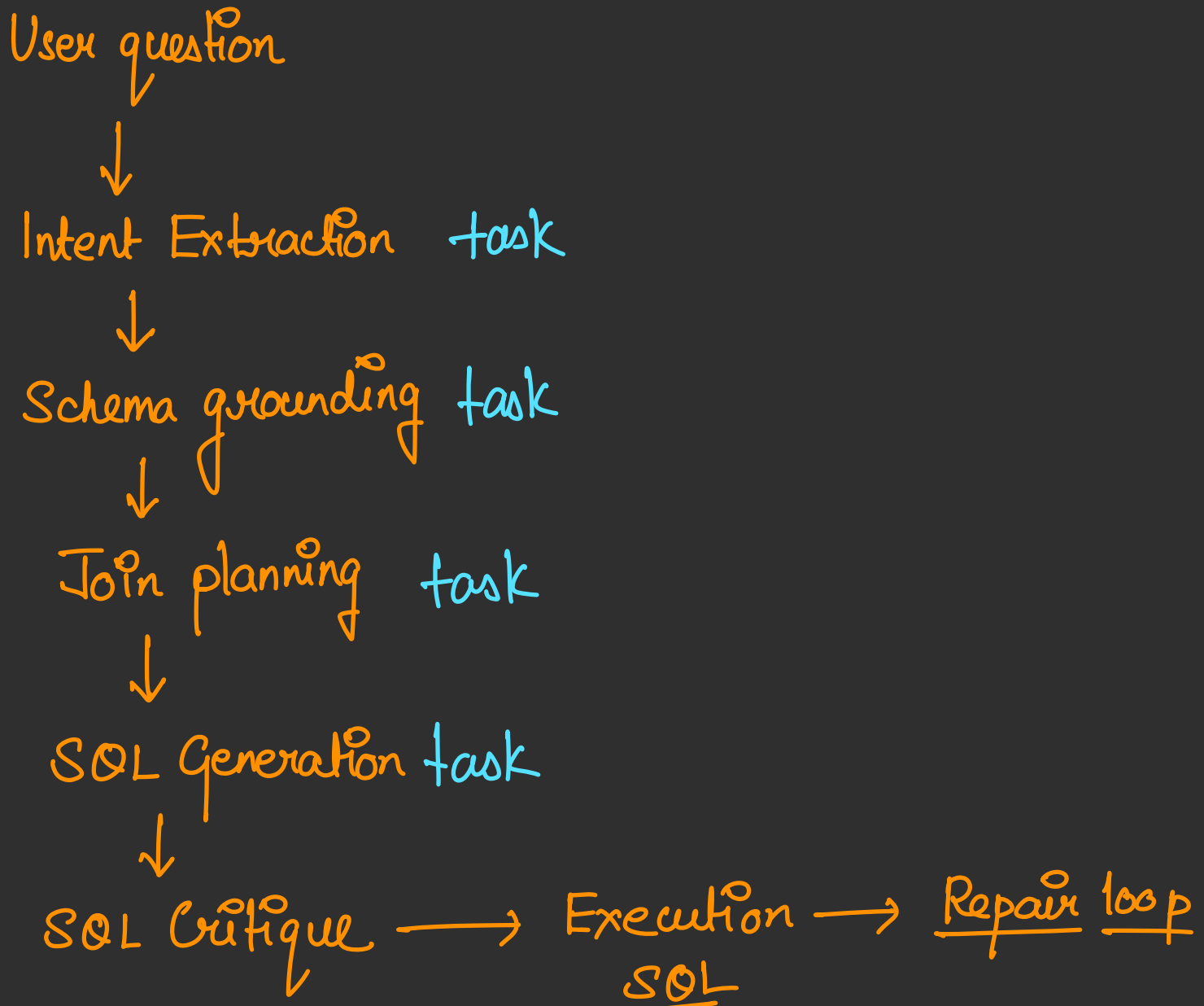
Agentic workflows close the loop: generate, evaluate, and adapt. They trade additional compute for much higher reliability — and are the foundation for building production-grade LLM systems that can handle real-world complexity.

Agentic Workflow

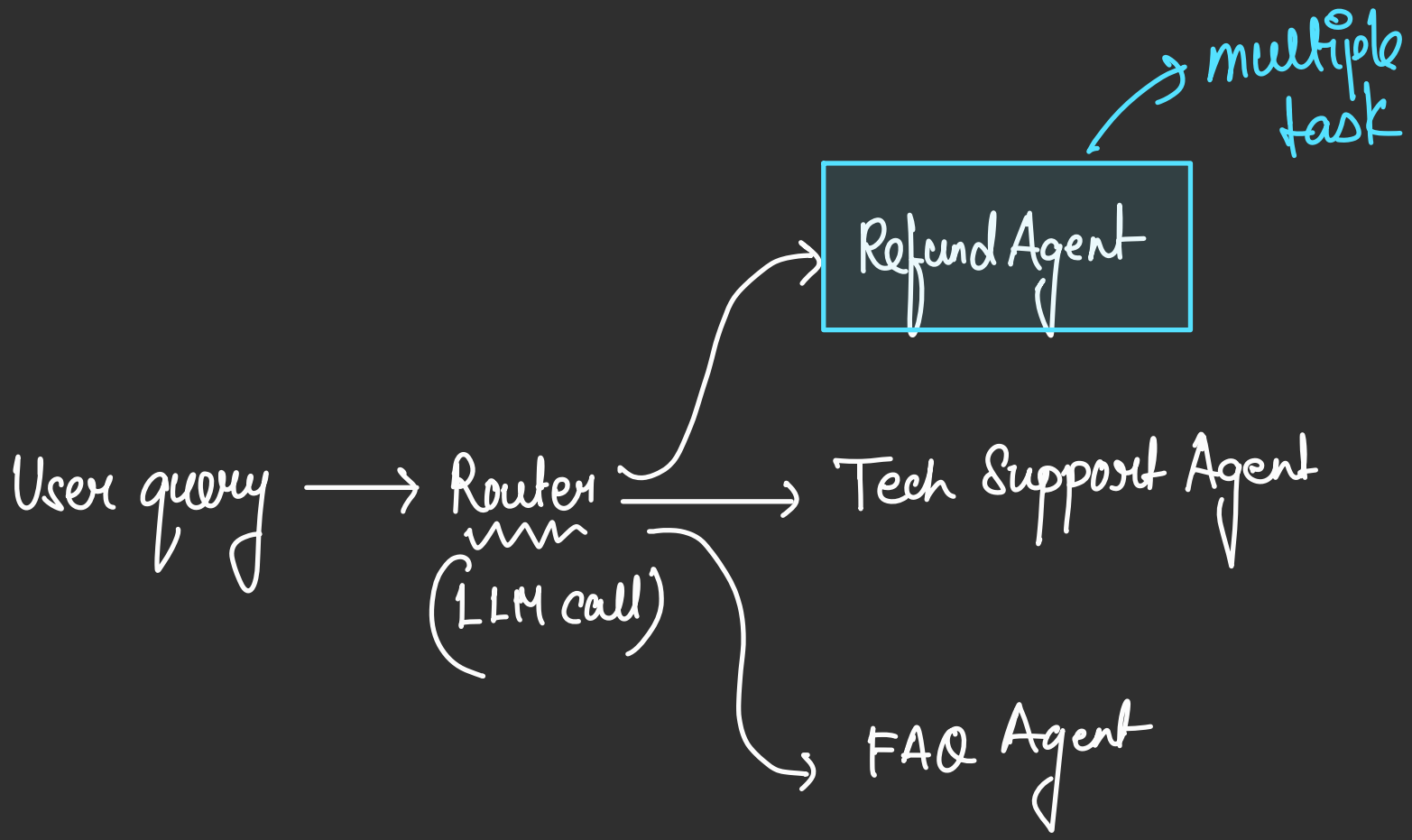
1) Chain prompting ✓
2) Routing ✓
3) Parallelization
4) Evaluator - Optimizer
5) Orchestrator - Worker

Chain prompting

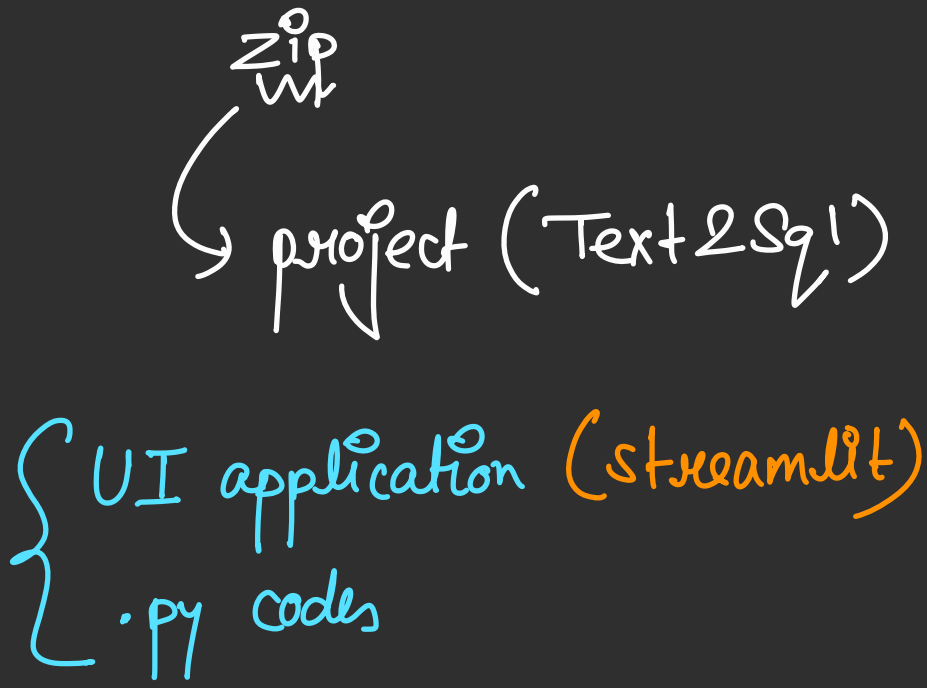



User question
Intent Extraction task
Schema grounding task
Join planning task
SQL Generation task
SQL Critique
Execution
SQL
Repair loop

#Routing



- Where is my order?
- I want a refund


zip
project (Text2Sql)
UI application (streamlit)
.py codes

goal: Not exactly to reduce hallucin-

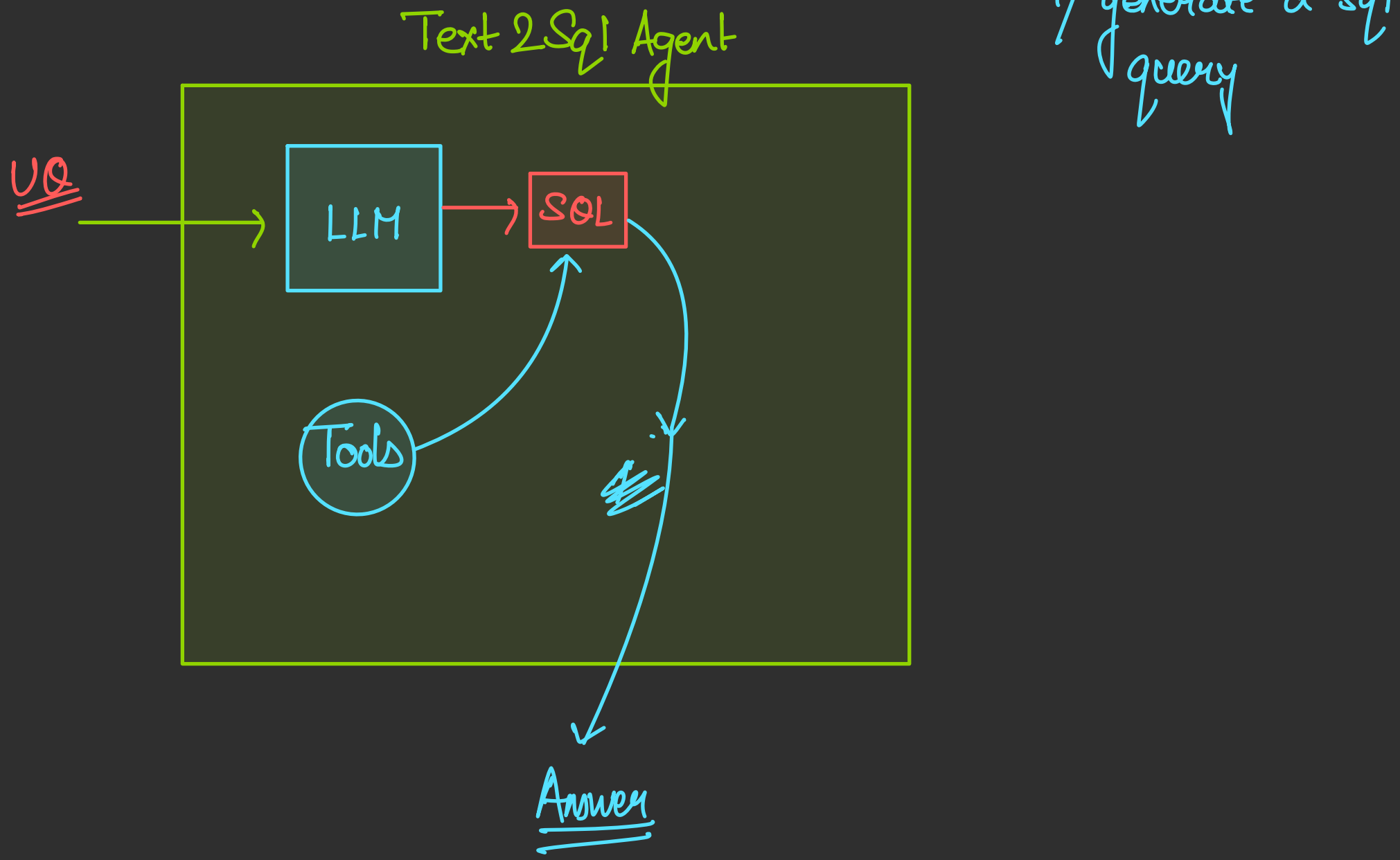
↓

break task

To reduce system error / improve system quality

5-7

1 LLM ⟶ 5-7 LLM



1) generate a sql query